

LLM 推理显存分析

目 录

1 引言：10 GiB 的硬约束	2
2 三部分显存构成	2
3 权重显存	2
3.1 公式	2
3.2 代入 Gemma4-12B 参数	3
4 KV Cache 显存	3
4.1 公式	3
4.2 随 batch 线性增长	3
5 算术强度与显存瓶颈	4
5.1 算术强度	4
5.2 显存瓶颈	4
6 静态分配与动态分配	4
6.1 静态分配	4
6.2 动态分配	4
7 W4A16：权重量化与激活保持	4
7.1 为什么不量化激活	5
7.2 反量化计算流程	5
8 显存优化方向	5
9 KVCache 实现	5
10 实战：10 GiB 显存预算分析	6
11 本章你将学会	6
12 小结	7

1 引言：10 GiB 的硬约束

Lab5 的核心约束是显存：我们只有 1/7 张 H800（MIG 10G），可用显存仅 10 GiB。Gemma4-12B 在 BF16 下权重就需要约 24 GiB，是显存上限的 2.4 倍。不解决显存问题，连模型都加载不了，更谈不上推理优化。

要优化显存，首先要知道它花在了哪里。本章我们拆解 LLM 推理时的三部分显存，推导各自的计算公式，理解为什么 INT4 量化是必选项，以及如何在此基础上进一步压缩 KV Cache。

2 三部分显存构成

LLM 推理时的显存由三部分组成：

部分	生命周期	特点
模型权重	整个服务期	固定，不随请求变化
KV Cache	请求生命周期	随 batch 和序列长度增长
激活值	单步计算	临时分配，算子完成后释放

直觉 把显存想象成一间厨房：权重是灶台和冰箱，一旦摆好就不再移动；KV Cache 是正在做的菜，每桌客人（batch）的菜占一份空间；激活值是切菜板上的食材，切完就收走。厨房面积有限，三类东西要合理安排。

权重显存在模型加载时确定，是显存占用的“地基”。KV Cache 随 batch size 和序列长度线性增长，是吞吐量的主要限制因素。激活值在算子执行时临时分配，通常通过算子融合来减少。

3 权重显存

3.1 公式

权重显存由嵌入层和 N 个 Transformer 层组成。每个层的参数包括 Q/O 投影、K/V 投影和 FFN 三个线性层。总参数量为：

$$M_w = \left[2VD + ND(2H_qD_h + 2H_{l,kv}D_{l,h} + 2H_{g,kv}D_{g,h} + 3F) \right] \cdot \frac{x}{8} \text{ bytes} \tag{3.1}$$

其中 x 是权重的量化位宽（BF16 时 $x = 16$ ，INT4 时 $x = 4$ ）， $\frac{x}{8}$ 把比特转换为字节。公式中 $2VD$ 是嵌入层（输入嵌入 + 输出投影）， $ND(...)$ 是 N 个层的参数总和。

各项的含义：

项	含义
$2VD$	嵌入层：输入嵌入和输出投影，各 $V \times D$
$2H_qD_h$	Q 和 O 投影，各 $D \times H_qD_h$
$2H_{l,kv}D_{l,h}$	滑动窗口层 K 和 V 投影
$2H_{g,kv}D_{g,h}$	全局层 K 和 V 投影
$3F$	FFN 的 gate、up、down 三个投影，各 $D \times F$

公式把滑动窗口层和全局层的 KV 参数都乘以 N ，实际上一部分层是滑动窗口、一部分是全局，这里做了简化估算。全局层 $W_K = W_V$ 时 KV 参数还会减半，但嵌入层占比不大，整体估计的误差在可接受范围内。

3.2 代入 Gemma4-12B 参数

用 $V = 262144$, $D = 3840$, $N = 48$, $H_q = 16$, $D_h = 256$, $H_{l,kv} = 8$, $D_{l,h} = 256$, $H_{g,kv} = 1$, $D_{g,h} = 512$, $F = 15360$:

嵌入层参数: $2 \times 262144 \times 3840 \approx 20.1$ 亿

每层参数: $3840 \times (8192 + 4096 + 1024 + 46080) = 3840 \times 59392 \approx 2.28$ 亿

总参数: $20.1 + 48 \times 2.28 \approx 129.6$ 亿

精度	位宽 x	权重大小
BF16	16	≈ 24.1 GiB
INT4	4	≈ 6.0 GiB

BF16 的 24 GiB 远超 10 GiB 限制，INT4 的 6 GiB 留出 4 GiB 给 KV Cache 和激活值，基本可行。

4 KV Cache 显存

4.1 公式

KV Cache 的显存占用公式为:

$$M_{kv} = \frac{2BSNH_{kv}D_h \cdot y}{8} \text{ bytes} \quad (4.1)$$

其中 y 是激活值位宽（推理时通常为 BF16, $y = 16$ ）。 B 是 batch size, S 是最大序列长度。2 是因为每个 token 既要存 Key 又要存 Value。

这里 H_{kv} 和 D_h 使用滑动窗口层的参数（8 头、256 维），忽略了全局层 KV 头更少但维度更大的差异。全局层只有 1 个 KV 头、512 维，与滑动层的 $8 \times 256 = 2048$ 相比， $1 \times 512 = 512$ 小得多，简化后整体偏大一点点。

4.2 随 batch 线性增长

KV Cache 是显存中唯一随 batch 线性增长的部分。固定 $S = 2048$ ，每增加一个 batch 槽位，KV Cache 增加:

$$\Delta M_{kv} = \frac{2 \times 1 \times 2048 \times 48 \times 8 \times 256 \times 16}{8} = 805,306,368 \text{ bytes} \approx 768 \text{ MiB} \quad (4.2)$$

每个 batch 槽位需要约 768 MiB 的 KV Cache。在 4 GiB 的可用空间内，最多容纳约 5 个 batch 槽位。

5 算术强度与显存瓶颈

5.1 算术强度

算术强度 (arithmetic intensity) 定义为每字节访存完成的计算量，单位是 FLOPS/byte。对于线性层 $Y = XW^T$ ：

$$\text{Arithmetic Intensity} = \frac{\text{FLOPS}}{\text{Bytes}} = \frac{2 \cdot \text{tokens} \cdot P}{P \cdot \frac{x}{8}} = 16 \cdot \frac{\text{tokens}}{x} \quad (5.1)$$

其中 P 是权重参数量，tokens 是当前处理的 token 数 (prefill 时为 S ，decode 时为 1)， x 是权重位宽。

- **Prefill**: tokens = S ，算术强度 = $16 \frac{S}{x}$ 。 $S = 1024$ 、 $x = 4$ 时为 4096 FLOPS/byte，远高于 GPU 的计算访存比，**计算密集**。
- **Decode**: tokens = 1，算术强度 = $\frac{16}{x}$ 。 $x = 4$ 时仅 4 FLOPS/byte，远低于 GPU 的计算访存比，**访存密集**。

直觉 Prefill 像批发：一次买 S 件货，运费均摊到每件很低；Decode 像零售：每次买 1 件，运费和货款一样贵。算术强度就是“货款/运费”比，比值高时计算吃得饱，比值低时在等快递。

5.2 显存瓶颈

Decode 阶段每个 token 都要读取整个模型的权重。INT4 下权重约 6 GiB，而 H800 MIG 的显存带宽有限，权重读取成为 decode 的主要瓶颈。这就是为什么后续优化中我们会做算子融合：在反量化的同时直接计算，避免把中间结果写入显存再读回。

6 静态分配与动态分配

6.1 静态分配

框架默认采用静态分配：按 `max_batch_size times max_sequence_length` 预分配连续的 KV Cache。优点是内存连续，访问效率高，不需要动态分配的开销。缺点是实际使用率可能很低：如果当前只有 1 个请求且序列长度只有 100，预分配的空间仍然占用全部显存。

静态分配的另一个问题是碎片：多个请求的 KV Cache 拼在一起，中间可能有空洞。这正是 *Paged Attention* 等技术要解决的。

6.2 动态分配

动态分配按需分配 KV Cache，请求到达时分配，完成后释放。优点是显存利用率高，缺点是分配/释放有开销，且可能产生碎片。*Paged Attention* 把 KV Cache 分成固定大小的页，按需分配，在利用率和效率之间取得平衡。

7 W4A16：权重量化与激活保持

W4A16 是本实验的量化方案：权重 INT4 (4 bit)，激活 BF16 (16 bit)。

7.1 为什么不量化激活

激活值的范围随输入变化，难以找到稳定的量化参数。更重要的是，decode 阶段每次只处理 1 个 token，激活值的数据量很小 ($1 \times D$)，量化激活节省的显存微乎其微，但带来的精度损失和反量化开销却不小。因此 W4A16 只量化权重，激活保持高精度。

7.2 反量化计算流程

W4A16 下，线性层的计算流程为：

1. 从显存读取 INT4 打包权重和 scales。
2. 在寄存器中反量化： $w = s \cdot (q - 8)$ 。
3. 用 BF16 激活值与反量化后的权重做矩阵乘。
4. 累加到 FP32 accumulator 后输出。

如果不做算子融合，反量化会先写入一个临时 BF16 张量，再被 GEMM 读回。这个临时张量的大小等于完整权重的 BF16 版本（约 24 GiB），显然不可行。算子融合把反量化和 GEMM 合并到一个 kernel 中，在寄存器中完成反量化后立即参与计算，不落盘。

8 显存优化方向

理解了三部分显存后，优化方向就清晰了：

优化目标	方法	本实验采用
权重显存	量化、offloading	INT4 量化 + 异步 offloading
KV Cache 显存	Paged Attention、Ring KV	静态分配 + 调度优化
激活显存	算子融合	反量化-GEMM 融合

权重 offloading 是 lab5 的关键优化：Gemma4 的 48 层按顺序执行，计算第 i 层时只需第 i 层权重在 GPU 上。其余层保存在 CPU 内存中，在执行前异步搬运到 GPU。这样 GPU 上只需保留 1-2 层的权重（约 0.5 GiB），把更多显存让给 KV Cache，从而支持更大的 batch size。

9 KVCache 实现

框架中 LayerKVCache 的形状为 [max_batch, kv_heads, max_seq_len, head_dim]：

```
class LayerKVCache:
    def __init__(self, max_batch, kv_heads, max_seq_len, head_dim, dtype):
        self.key = torch.empty(
            max_batch, kv_heads, max_seq_len, head_dim, dtype=dtype
        )
        self.value = torch.empty(
            max_batch, kv_heads, max_seq_len, head_dim, dtype=dtype
        )
        self.seq_len = 0
```

滑动窗口层和全局层的 KV Cache 形状不同：

层类型	kv_heads	head_dim
滑动窗口层	8	256
全局层	1	512

全局层虽然 head_dim 更大，但只有 1 个 KV 头，总 KV Cache 更小（ 1×512 vs 8×256 ）。

10 实战：10 GiB 显存预算分析

我们做一个完整的显存预算分析，计算 10 GiB 下能支持的最大 batch size。

例 已知： INT4 权重约 6.0 GiB， $S = 2048$ ，BF16 激活。

第 1 步：KV Cache 单 batch 占用

$$M_{\text{kv,per batch}} = \frac{2 \times 1 \times 2048 \times 48 \times 8 \times 256 \times 16}{8} = 805,306,368 \text{ bytes} \approx 768 \text{ MiB} \quad (10.1)$$

第 2 步：可用显存

$$\text{可用} = 10 - 6.0 = 4.0 \text{ GiB} \quad (10.2)$$

扣除 CUDA 上下文和激活值临时显存（约 0.5 GiB）：

$$\text{KV Cache 可用} = 4.0 - 0.5 = 3.5 \text{ GiB} \quad (10.3)$$

第 3 步：最大 batch size

$$\text{max batch} = \left\lfloor 3.5 \times \frac{1024}{768} \right\rfloor = \lfloor 4.67 \rfloor = 4 \quad (10.4)$$

所以在 10 GiB 显存下，INT4 量化 + 静态分配可以支持 batch size 最大为 4。

第 4 步：offloading 后

如果实现权重 offloading，GPU 上只保留 2 层权重：

$$M_{\text{w,GPU}} = 2 \times (3840 \times 59392) \times \frac{4}{8} = 2 \times 228,065,280 \times 0.5 = 228,065,280 \text{ bytes} \approx 218 \text{ MiB} \quad (10.5)$$

释放的显存： $6.0 - 0.218 \approx 5.78 \text{ GiB}$

$$\text{KV Cache 可用} = 10 - 0.218 - 0.5 = 9.28 \text{ GiB} \quad (10.6)$$

$$\text{max batch} = \left\lfloor 9.28 \times \frac{1024}{768} \right\rfloor = \lfloor 12.4 \rfloor = 12 \quad (10.7)$$

offloading 后理论上可支持 batch size 12，是未 offloading 时的 3 倍。实际受 CPU-GPU 带宽限制和调度开销影响，可能达不到理论值，但 batch size 的提升幅度显著。

这个分析说明为什么 offloading 是 lab5 中效果最显著的优化：它把权重显存从 6 GiB 降到 0.2 GiB，释放的空间全部转化为 KV Cache 容量，直接提升 batch size 和吞吐量。

11 本章你将学会

1. 列出 LLM 推理显存的三个组成部分，说明各自的生命周期和特点。
2. 代入 Gemma4-12B 参数计算权重显存，验证 BF16 约 24 GiB、INT4 约 6 GiB。
3. 推导 KV Cache 显存公式，计算单 batch 在 $S = 2048$ 时的占用。
4. 解释算术强度的定义，说明 prefill 计算密集、decode 访存密集的原因。
5. 描述 W4A16 方案的反量化计算流程，说明为什么需要算子融合。

6. 在 10 GiB 显存预算下计算最大 batch size, 分析 offloading 对 batch size 的提升。

12 小结

LLM 推理显存由权重、KV Cache 和激活值三部分组成。INT4 量化把权重从 24 GiB 压缩到 6 GiB, 是 10 GiB 显存约束下的必选项。KV Cache 随 batch 线性增长, 每个 batch 在 $S = 2048$ 时占 768 MiB, 是限制吞吐量的主要因素。算术强度分析揭示了 decode 阶段的访存瓶颈, W4A16 方案在保持计算精度的同时压缩权重存储。Offloading 把权重显存从 6 GiB 降到 0.2 GiB, 将空间让给 KV Cache, 是提升 batch size 和吞吐量的关键手段。

讲义基于 HPC Lab5 实验指导编写